In [ ]:

```
# REAL-WORLD DATA PREPROCESSING PIPELINE          Pragya Singhal CU24220050

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Set visualization style           Pragya Singhal CU24220050
sns.set_theme(style="whitegrid")

# Step 1 & 3: Load the Dataset & Inspect Structure
print("--- 1. LOADING DATASET ---")
# Using the public Seaborn repository for a reliable Titanic dataset source
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic
df = pd.read_csv(url)

print("\nFirst 5 rows of the dataset:")
display(df.head())

print("\nDataset Information:")
print(df.info())

print("\nStatistical Summary:")
display(df.describe())

# Step 4: Handle Missing Values
print("\n--- 2. HANDLING MISSING VALUES ---")
print("Missing values before treatment:")
print(df.isnull().sum())

# Age: Fill missing values with the median age
df['Age'] = df['Age'].fillna(df['Age'].median())

# Embarked / Embarked_Town: Fill with mode (most frequent value)
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

# Cabin: Too many missing values, drop the column or create a flag. Let's drop i
df = df.drop(columns=['Cabin'])

# Drop 'PassengerId' and 'Name' as they aren't useful for general modeling
df = df.drop(columns=['PassengerId', 'Name', 'Ticket'])

print("\nMissing values after treatment:")
print(df.isnull().sum())

# Step 5: Remove Duplicate Records                Pragya Singhal CU24220050
print("\n--- 3. REMOVING DUPLICATES ---")
initial_rows = len(df)
df = df.drop_duplicates()
removed_rows = initial_rows - len(df)
print(f"Removed {removed_rows} duplicate rows.")

# Step 6: Standardize & Encode Categorical Variables            Pragya Singhal
print("\n--- 4. ENCODING CATEGORICAL VARIABLES ---")
# Check unique values in categorical columns to ensure consistency
print("Unique Sex values:", df['Sex'].unique())
print("Unique Embarked values:", df['Embarked'].unique())
```

```python
# Convert 'Sex' using Binary / Label Encoding (Male=1, Female=0)
df['Sex'] = df['Sex'].map({'male': 1, 'female': 0})

# Convert 'Embarked' using One-Hot Encoding
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)

# Ensure boolean columns from get_dummies are converted to integers (0 or 1)
bool_cols = df.select_dtypes(include=['bool']).columns
df[bool_cols] = df[bool_cols].astype(int)

print("\nData types after encoding:")
print(df.dtypes)

# Step 7: Outlier Detection and Treatment (IQR Method)
print("\n--- 5. OUTLIER TREATMENT ---")
# Let's check outliers in 'Fare' using the IQR method
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print(f"Fare IQR bounds: Lower = {lower_bound}, Upper = {upper_bound}")

# Cap the outliers (Winsorization) instead of dropping rows to retain data
df['Fare'] = np.where(df['Fare'] > upper_bound, upper_bound, df['Fare'])
df['Fare'] = np.where(df['Fare'] < lower_bound, lower_bound, df['Fare'])
print("Outliers in 'Fare' have been capped successfully.")

# Step 8: Normalization / Standardization             Pragya Singhal CU24220050
print("\n--- 6. FEATURE NORMALIZATION ---")
# Standardizing continuous features ('Age' and 'Fare') using Z-score normalizati
for col in ['Age', 'Fare']:
    mean_val = df[col].mean()
    std_val = df[col].std()
    df[f'{col}_Scaled'] = (df[col] - mean_val) / std_val

print("Features 'Age' and 'Fare' standardized.")

# Step 9: Feature Engineering                  Pragya Singhal CU24220050
print("\n--- 7. FEATURE ENGINEERING ---")
# Create 'FamilySize' from SibSp (Siblings/Spouses) and Parch (Parents/Children)
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1

# Create an 'IsAlone' binary flag
df['IsAlone'] = np.where(df['FamilySize'] == 1, 1, 0)

print("\nEngineered features added: 'FamilySize', 'IsAlone'")
display(df[['SibSp', 'Parch', 'FamilySize', 'IsAlone']].head())

# Step 10: Save Cleaned Dataset
print("\n--- 8. SAVING CLEANED DATASET ---")
output_filename = "cleaned_titanic_dataset.csv"
df.to_csv(output_filename, index=False)
print(f"Success! Cleaned dataset saved to '{output_filename}'. Total rows: {len(
```

```
--- 1. LOADING DATASET ---

First 5 rows of the dataset:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fa
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25(
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.28:
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.92!
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.10(
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05(

```
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
None

Statistical Summary:
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fa
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.0000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.2042
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.6934
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.0000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.9104
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.4542
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.0000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.3292

```
--- 2. HANDLING MISSING VALUES ---
Missing values before treatment:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64

Missing values after treatment:
Survived    0
Pclass      0
Sex         0
Age         0
SibSp       0
Parch       0
Fare        0
Embarked    0
dtype: int64

--- 3. REMOVING DUPLICATES ---
Removed 116 duplicate rows.

--- 4. ENCODING CATEGORICAL VARIABLES ---
Unique Sex values: <StringArray>
['male', 'female']
Length: 2, dtype: str
Unique Embarked values: <StringArray>
['S', 'C', 'Q']
Length: 3, dtype: str

Data types after encoding:
Survived        int64
Pclass          int64
Sex             int64
Age           float64
SibSp           int64
Parch           int64
Fare          float64
Embarked_Q      int64
Embarked_S      int64
dtype: object

--- 5. OUTLIER TREATMENT ---
Fare IQR bounds: Lower = -31.171850000000003, Upper = 73.41975000000001
Outliers in 'Fare' have been capped successfully.

--- 6. FEATURE NORMALIZATION ---
Features 'Age' and 'Fare' standardized.

--- 7. FEATURE ENGINEERING ---

Engineered features added: 'FamilySize', 'IsAlone'
```

	SibSp	Parch	FamilySize	IsAlone
0	1	0	2	0
1	1	0	2	0
2	0	0	1	1
3	1	0	2	0
4	0	0	1	1

```
--- 8. SAVING CLEANED DATASET ---
Success! Cleaned dataset saved to 'cleaned_titanic_dataset.csv'. Total rows: 775,
Total columns: 13
```